



Cours 6 : Matrice

La liste en Python n'accepte pas que les types de bases, il est également possible de stocker des types construits dans une liste. Ainsi, il est possible d'imaginer la construction d'une structure en 2 dimensions : une matrice, créée à partir d'une liste de listes.

```
# création de 3 listes
liste_1 = [1,2,3]
liste_2 = [4,5,6]
liste_3 = [7,8,9]

# création d'une liste contenant d'autres listes : une matrice
matrice = [liste_1, liste_2, liste_3]
print(matrice)
>> [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

# construction par compréhension
matrice = [[i] for i in range(10)]
print(matrice)
>> [[0], [1], [2], [3], [4], [5], [6], [7], [8], [9]]
```

Accès aux valeurs

Pour accéder aux différentes valeurs d'une matrice, il faut utiliser des []. Notre matrice étant une "grande" liste de "petites" listes, on peut accéder aux petites listes comme si on accédait à un élément quelconque dans une liste à 1 dimension. Ensuite, quand on a choisi quelle petite liste on voulait atteindre, on peut elle même la parcourir avec de nouveaux [].

```
# création d'une matrice
matrice = [[1,2,3], [4,5,6], [7,8,9]]

# taille d'une matrice
print(len(matrice))
>> 3

# accès à la première petite liste de ma grande liste
print(matrice[0])
>> [1, 2, 3]

# accès dans la première petite liste à son troisième élément
print(matrice[0][2])
>> 3
```

Parcours d'une matrice

Si je veux modifier ou rechercher un élément dans une matrice, je vais devoir la parcourir. Pour se faire, comme il s'agit d'une liste de listes je vais avoir besoin de deux indices : un premier indice pour me déplacer de liste en liste, puis un second indice pour parcourir les éléments de la liste que je regarde actuellement.

```
# création d'une matrice contenant les notes des deux groupes de NSI
notes = [[17, 5, 12, 3, 12, 9], [9, 17, 18, 15, 12]]

# je cherche à rajouter 2 points à tous les élèves

for i in range(len(notes)): # je parcours chaque liste de ma matrice
    for j in range(len(notes[i])): # je parcours chaque élément de la liste actuelle
        notes[i][j] += 2 # je change la note de l'élève j du groupe i
```

```
print(notes)
>> [[19, 7, 14, 5, 14, 11], [11, 19, 20, 17, 14]]
```

Pour résumer, on peut stocker dans un tableau d'autres types construits. On peut accéder avec une seule paire de `[]` à l'élément de notre choix de la liste peu importe son type, puis si l'élément choisi est lui même un type construit, alors on peut de nouveau accéder à ses sous éléments, etc. Une matrice est une liste de listes (ou liste à deux dimensions) où toutes les petites listes font la même taille.



Activité 20 : Loop I choose you

Écrire une fonction qui prend en paramètre une matrice et une valeur à rechercher dans la matrice, la fonction renvoie True si l'élément est présent, False sinon. Par exemple :

```
matrice_caracteres = [['a', 'e', 'i', 'o', 'u'], ['t', 'r', 'p', 'l', 's']]
recherche_car(matrice_caracteres, 'u')
>> True
recherche_car(matrice_caracteres, 'z')
>> False
```

Écrire cette fonction sous deux formes différentes, l'une avec une boucle while et la seconde avec une boucle for. D'après vous, quelle méthode est la plus pertinente pour effectuer une recherche ? Pourquoi ?

Modifier ensuite vos fonctions pour qu'elles renvoient également, si l'élément est présent, la position de l'élément, sous la forme d'un p-uplet. Par exemple :

```
matrice_caracteres = [['a', 'e', 'i', 'o', 'u'], ['t', 'r', 'p', 'l', 's']]
recherche_car(matrice_caracteres, 'u')

#Ici, l'élément est présent, plus particulièrement en
#cinquième position dans la première petite liste
>> True, (0, 4)

recherche_car(matrice_caracteres, 'z')
>> False
```



Activité 21 : Rupture

Écrire une fonction Python qui prend en paramètre une liste contenant des valeurs numériques qui renvoie une liste de liste. La première liste contiendra les valeurs paires et la seconde les impaires. Par exemple :

```
separation([4, 8, 9, 6, 435])
>> [[4, 8, 6], [9, 435]]
```



Cours 7 : Méthodes sur les listes

Il existe sur les listes des **méthodes** permettant aux utilisateurs de les manipuler plus facilement sous le format `nom_liste.methode(paramètres)`. Voici un panel des méthodes disponibles sur les listes que vous pouvez utiliser ou rencontrer dans un programme :

```
ma_liste.append(valeur)
ma_liste.insert(position, valeur)
ma_liste.remove(valeur)
ma_liste.pop(position)
ma_liste.count(valeur)
ma_liste.index(valeur)
```

```
permet d'ajouter un élément à la fin de la liste
permet d'ajouter un élément à une position précise
supprime le premier élément égal à valeur présent dans la liste
supprime l'élément à la position donnée et renvoie sa valeur
compte le nombre d'occurrences d'un élément dans une liste
renvoie la position de la première occurrence de l'élément égal à valeur
```

Il est possible de retrouver l'ensemble des méthodes en utilisant la commande `help(list)` dans votre interpréteur Python.



Activité 22 : En somme

Écrire une fonction Python qui prend en paramètres deux matrices et qui les additionne. Si l'une des matrices est plus grande que la seconde, sa valeur sera additionnée avec 0. Par exemple :

```
premiere_matrice = [[1, 2], [3, 4]]
seconde_matrice = [[7, 8, 9], [6, 14, 29]]
somme_matrice(premiere_matrice, seconde_matrice)
>> [[8, 10, 9], [9, 18, 29]]
```



Activité 23 : Math'rice

Écrire une fonction qui prend en paramètres une matrice d'entiers et un entier n , et qui vérifie si la somme des valeurs de la colonne n est égale à la somme de la ligne n . Par exemple :

```
matrice_entiers = [[1, 2, 5], [3, 4, 9], [4, 7, 7]]
verif_matrice(matrice_entiers, 0)
>> True
verif_matrice(matrice_entiers, 2)
>> False
```